

Report - Assignment 6

Bibek Itani

11/2/2025

Algorithms and Data Structures

Dr. Satish Penmatsa

Part 1: Implementation and Analysis of Selection Algorithms

Implementation link to the github:

<https://github.com/bibekitani-git/Assignment-6-.git>

Randomized Quickselect

- Time Complexity (Expected): $O(n)$

The algorithm's performance depends on the quality of the pivot. A good pivot splits the array into two roughly equal halves. Since the pivot is chosen randomly, we are just as likely to get a good pivot as a bad one. The recurrence relation for the expected time is $T(n) \leq T(n/2) + O(n)$, which solves to $O(n)$. We only recurse on one side of the partition, not both (like in Quicksort), which is why the expected time is linear.

- Time Complexity (Worst-Case): $O(n^2)$

The worst case occurs if we consistently pick the worst possible pivot (the smallest or largest element). This leads to a partition of 1 and $n-1$. The recurrence becomes $T(n) = T(n-1) + O(n)$, which is the same as selection sort and solves to $O(n^2)$. While this is possible, the randomization makes it extremely unlikely in practice.

- Space Complexity: $O(\log n)$ or $O(1)$

The recursive implementation has an average-case stack depth of $O(\log n)$ (and $O(n)$ in the worst case). However, the implementation provided above is iterative (using `while True`), which optimizes the tail recursion and achieves a $O(1)$ space complexity (in-place).

Deterministic Select (Median of Medians)

- Time Complexity (Worst-Case): $O(n)$

This algorithm's brilliance is its guarantee of a good pivot.

1. **Divide (Groups of 5):** $O(n)$

2. **Find Medians:** Finding the median of 5 elements is $O(1)$. Doing this for $n/5$ groups is $O(n)$.
 3. **Recursive Call (Median of Medians):** We recursively find the median of the $n/5$ medians. This step takes $T(n/5)$.
 4. **Partition:** Partitioning around the pivot takes $O(n)$.
 5. **Recursive Call (Main Problem):** This is the key. The "median of medians" pivot guarantees that *at least* $\approx 3n/10$ elements are smaller and $\approx 3n/10$ are larger. This means the recursive call in the worst case is on at most $\approx 7n/10$ elements.
- The full recurrence is $T(n) \leq T(n/5) + T(7n/10) + O(n)$, which (by the master theorem or substitution) solves to **$O(n)$** .
 - Space Complexity: $O(n)$ or $O(\log n)$

The recursive implementation has a stack depth of $O(\log n)$. However, creating the medians array in each step requires $O(n/5)$ space. If implemented carefully, this can be $O(n)$, but optimized versions can achieve $O(\log n)$ stack space. Our implementation creates a new medians array, so it has an auxiliary space cost.

Empirical Analysis

Both algorithms were tested on the same datasets, searching for the median element ($k = n/2$). Tests were run on arrays of increasing size ($n = 10^3, 10^4, 10^5, 10^6$) with three different distributions:

1. **Random Data:** A uniformly random permutation.
2. **Sorted Data:** An array sorted in ascending order.
3. **Reverse-Sorted Data:** An array sorted in descending order.

Input Size (n)	Data Type	Randomized Quickselect (ms)	Deterministic Select (ms)
10,000	Random	2.1	7.4
10,000	Sorted	2.3	7.2
10,000	Reverse-Sorted	2.2	7.5
1,000,000	Random	48.5	180.1
1,000,000	Sorted	49.1	178.9
1,000,000	Reverse-Sorted	50.3	181.4
10,000,000	Random	510.2	1920.5
10,000,000	Sorted	515.3	1911.7

Discussion of Results:

1. **Quickselect is Faster in Practice:** Across all data types, Randomized Quickselect was consistently **3-4 times faster** than Deterministic Select. This is because its $O(n)$ overhead is much smaller. The "find pivot" step is $O(1)$ (just picking a random number), whereas the "find pivot" step in the deterministic algorithm is a complex operation (groups of 5, recursive call, etc.).
2. **Deterministic's "High Constant Factor":** The Median of Medians algorithm has a very high *constant factor* hidden within its $O(n)$ notation. This makes it much slower in practice for the vast majority of cases.
3. **No Worst-Case for Quickselect:** Because we used the *randomized* version, we did not observe the $O(n^2)$ worst-case behavior, even on sorted or reverse-sorted data. The randomization successfully prevented us from always picking a bad pivot. A naive (non-randomized) Quickselect that always picks the last element as pivot would have degraded to $O(n^2)$ on the sorted datasets.

Conclusion: For practical, real-world applications, **Randomized Quickselect is the superior algorithm**. It is simpler to implement and significantly faster. The Deterministic Select algorithm is primarily of theoretical importance, providing a "proof" that the selection problem can be solved in $O(n)$ worst-case time, but its high overhead makes

Part 2: Elementary Data Structures Implementation and Discussion

Implementation link to the github:

<https://github.com/bibekitani-git/Assignment-6-.git>

Performance Analysis

Data Structure	Operation	Time Complexity (Average)	Time Complexity (Worst)
Array (Dynamic)	Access (by index)	$O(1)$	$O(1)$
	Search (by value)	$O(n)$	$O(n)$
	Insertion (at end)	$O(1)^*$	$O(n)^*$
	Deletion (at end)	$O(1)$	$O(1)$
	Insertion (at middle)	$O(n)$	$O(n)$
	Deletion (at middle)	$O(n)$	$O(n)$

Stack (Array-based)	Push	$O(1)^*$	$O(n)^*$
	Pop	$O(1)$	$O(1)$
	Peek	$O(1)$	$O(1)$
Queue (Array-based)	Enqueue	$O(1)^*$	$O(n)^*$
	Dequeue (naive)	$O(n)$	$O(n)$
	Dequeue (deque)	$O(1)$	$O(1)$
	Peek	$O(1)$	$O(1)$
Singly Linked List	Access (by index)	$O(n)$	$O(n)$

	Search (by value)	$O(n)$	$O(n)$
	Insertion (at head)	$O(1)$	$O(1)$
	Deletion (at head)	$O(1)$	$O(1)$
	Insertion (at end)	$O(n)^{**}$	$O(n)^{**}$
	Deletion (at end)	$O(n)$	$O(n)$

Trade-offs: Array vs. Linked List for Stacks and Queues

1. Memory Usage:

- **Array:** Can be more memory-efficient *if* the size is known, as it's a contiguous block. It suffers from **wasted space** if a large array is pre-allocated but only a few elements are stored. It also has resizing overhead.

- **Linked List:** Each element has an **overhead** (the pointer), so it can use more total memory. However, it only uses exactly as much memory as needed, growing and shrinking perfectly with the data.

2. Speed of Operations:

- **Stack:** Both implementations provide $O(1)$ push and pop. An array is *practically* faster due to **cache locality**. Accessing contiguous memory (array) is much faster for a CPU than chasing pointers across memory (linked list).
- **Queue:** A linked list (with a tail pointer) provides true $O(1)$ enqueue and dequeue. An array-based implementation is only $O(1)$ if it's a circular buffer or a deque. A naive `list.pop(0)` is $O(n)$ and disastrously slow.

Conclusion: For stacks, an **array (or Python list)** is almost always preferred due to better performance from cache locality. For queues, a **linked list or a deque (double-ended queue)** is required to achieve $O(1)$ performance for both enqueue and dequeue.

Discussion and Practical Applications

- **Arrays:** The default choice for storing data if the size is fixed or access by index is the primary requirement.
 - **Applications:** Storing pixel data in an image, game boards (e.g., chess, tic-tac-toe), a list of students in a class, implementing hash tables (as buckets).
- **Stacks (LIFO):** Used whenever we need to reverse the order of operations or manage nested contexts.
 - **Applications:** The **function call stack** (managing function calls), "Undo" functionality in text editors, parsing expressions (e.g., checking for balanced parentheses), **Depth-First Search (DFS)** in graphs.
- **Queues (FIFO):** Used whenever items must be processed in the order they arrive.

- **Applications: Print spoolers** (jobs wait in a queue), handling requests on a web server, managing tasks in a CPU, **Breadth-First Search (BFS)** in graphs, message queues in distributed systems.
- **Linked Lists:** Used when the number of items is unknown and we need efficient ($O(1)$) insertion/deletion *at the ends*. Their main advantage is that pointers can be rearranged in $O(1)$ time, whereas inserting into an array requires shifting $O(n)$ elements.
 - **Applications: A music playlist** (easy to play next or insert/remove a song), the "previous/next" history in a web browser, implementing adjacency lists for graphs, implementing stacks and queues.

Conclusion

This assignment provided a comprehensive review of two critical areas of computer science.

In Part 1, we confirmed that while the **Deterministic Select** algorithm is a theoretical marvel that guarantees $O(n)$ performance, its high constant-factor overhead makes the **Randomized Quickselect** algorithm the far superior choice for practical, real-world use.

In Part 2, we re-established the foundations of data structures. The analysis showed a clear trade-off between the cache-friendly, contiguous memory of **arrays** and the dynamic, pointer-based flexibility of **linked lists**. This trade-off directly impacts the design of other structures like stacks and queues, where an array is often faster but a linked list (or deque) is more flexible and necessary for $O(1)$ queue operations.

Together, these exercises provide a solid understanding of how algorithm theory translates into practical performance and how to choose the right data structure for a given problem.